

WEB TECHNOLOGY

Lecture 02 & Lecture 03 — Detailed Study Notes

DNS • Internet • IP • TCP • TLS • HTTP • URLs • Caching • CDN • REST • APIs • Authentication

Purpose: Complete explanations of the concepts in the uploaded handwritten lecture, with definitions, examples, comparisons and technical clarifications. This is intentionally more detailed than the original flow diagrams.

Source basis: “Web Tech Lecture 2+3.pdf” — 5-page handwritten scan. The source uses shorthand in several places; where a point needs precision, it is labeled **Technical clarification** rather than silently changing the lecture.

DETAILED NOTES

Designed for understanding, revision and exam preparation.

Contents

1. Request Flow
2. Internet vs Web
3. Internet History
4. Networking Layers
5. IP Addresses
6. DNS and Domain Resolution
7. DNS Caching and TTL
8. TCP, Ports and Port Binding
9. TLS Handshake and HTTPS
10. CDN
11. URL Breakdown
12. API and HTTP Requests
13. HTTP Methods
14. Safe, Idempotent and Cacheable
15. HTTP Headers
16. HTTP Status Codes
17. Switching Protocols
18. Web Caching
19. REST
20. SOAP vs REST vs GraphQL
21. Authentication vs Authorization
22. Cookies, Sessions, JWT and Client State
23. WebSocket
24. React / Angular Note
25. End-to-End Example
26. Exam Revision
27. Technical Clarifications

1. Request Flow

The lecture's main diagram is approximately Browser → Resolver / Root → TLD → IP → TCP → HTTP / TLS → Response. It is a conceptual shortcut showing the major systems involved when a browser accesses a web resource.

Technical clarification: A more accurate modern flow is: URL parsing → browser/OS DNS cache checks → recursive DNS resolution if needed → IP address obtained → transport connection → TLS for HTTPS → HTTP request → HTTP response → browser processing/rendering/caching.

Detailed sequence

- 1 **URL parsing:** the browser identifies the scheme, host, port, path, query and fragment.
- 2 **DNS:** the hostname must be translated into an IP address. The answer may already be cached.
- 3 **Transport:** HTTP/1.1 and HTTP/2 normally use TCP; HTTP/3 uses QUIC over UDP.
- 4 **TLS:** HTTPS uses TLS to authenticate the server and protect the communication.
- 5 **HTTP request:** the client sends a method such as GET, plus headers and possibly a body.
- 6 **Server processing:** the application may use a database, cache, authentication and authorization.
- 7 **HTTP response:** the server returns a status code, headers and optionally a body.
- 8 **Client processing:** the browser/app displays, parses, stores or otherwise uses the response.

Example: <https://api.example.com/v1/students/42>

The browser resolves `api.example.com`, connects to the server, establishes TLS because the URL uses HTTPS, sends a GET request for `/v1/students/42`, and may receive 200 OK with JSON data.

2. Internet vs Web

Internet: a global interconnected network of networks. It provides the infrastructure and protocols through which devices communicate.

Web: an application/information system running on the Internet. Browsers, URLs, HTTP/HTTPS, web servers, HTML, CSS and JavaScript are part of the Web ecosystem.

Key idea: the Internet is not the same thing as the Web. The Web is one service that uses the Internet.

Internet	Web
Network of interconnected networks	Application/information system on the Internet
Many protocols: IP, TCP, UDP, DNS, etc.	Primarily HTTP/HTTPS plus web technologies
Carries many services	Websites and web applications

3. Internet History

The scan lists 1969, 1983, 1989 and 1993 as major milestones.

Year	Lecture shorthand	Precise explanation
1969	Internet	ARPANET reached an early four-node network and the first host-to-host message was sent. The modern Internet evolved over
1983	TCP/IP	January 1, 1983 is associated with ARPANET's transition to TCP/IP.
1989	Web	Tim Berners-Lee proposed the World Wide Web at CERN in 1989.
1993	First web browser	Mosaic was released in 1993 and helped popularize the graphical Web; it was not literally the first browser.

Why these dates matter: networking infrastructure, protocols, naming systems, browsers and web applications developed progressively rather than appearing as one finished system on one date.

4. Networking Layers

Layering separates responsibilities. The OSI model is a useful teaching model; real Internet protocols do not always fit exactly into one OSI layer.

OSI layer	Responsibility	Examples
7 Application	Application communication	HTTP, DNS, SMTP
6 Presentation	Representation, encoding, encryption/compression concepts	Encoding; TLS overlaps practical layers
5 Session	Logical session concepts	Often handled by applications today
4 Transport	Ports, reliability, ordering, congestion	TCP, UDP, QUIC
3 Network	Logical addressing/routing	IPv4, IPv6
2 Data Link	Local frames/MAC	Ethernet, Wi-Fi
1 Physical	Signals/bits	Fiber, copper, radio

Encapsulation

HTTP data → TLS records → TCP segment → IP packet → Ethernet/Wi-Fi frame

At the receiver the process is reversed. In the practical TCP/IP model, these responsibilities are often grouped into Link, Internet, Transport and Application layers.

5. IP Addresses — IPv4 and IPv6

IP (Internet Protocol) provides logical addressing and routing so packets can move between networks.

IPv4: 32 bits, normally written as four decimal octets. Example: 192.0.2.10.

IPv6: 128 bits, normally hexadecimal groups separated by colons. Example: 2001:db8::10.

Type/category	Meaning	Example
Public	Globally routable address	Provider-assigned public IP
Private IPv4	Internal address space	10.0.0.0/8; 172.16.0.0/12; 192.168.0.0/16
Loopback	Host talks to itself	127.0.0.1; ::1
Link-local	Local-link addressing	169.254.0.0/16; fe80::/10
Static	Intended to remain stable	Fixed server address
Dynamic	Automatically assigned/changed	DHCP address
Unicast	One sender → one destination	Typical web request
Multicast	One sender → subscribed group	Protocol/service discovery cases
Anycast	Same address announced from multiple locations	Distributed network services

IPv6: does not use IPv4-style broadcast; multicast is used for many one-to-many operations. **NAT** commonly lets many private IPv4 devices share one public IPv4 address.

6. DNS — Domain Name System

DNS is a distributed hierarchical naming system. It can return different record types; the most familiar are A and AAAA records that provide IPv4 and IPv6 addresses.

Browser cache → OS cache → Recursive Resolver → Root → TLD → Authoritative DNS

Components

- **Browser cache:** recent DNS information may be remembered by the browser.
- **OS cache:** the operating system may cache answers and also use local host configuration.
- **Recursive resolver:** performs DNS lookups for the client. It may be operated by an ISP, company or public DNS provider.

- **Root:** directs the resolver toward the relevant TLD service.
- **TLD:** such as .com, .org, .in or .gov; directs the resolver toward authoritative nameservers.
- **Authoritative nameserver:** provides authoritative records for a DNS zone.
- **A:** IPv4 address. **AAAA:** IPv6 address. **CNAME:** hostname alias. **MX:** mail server. **NS:** nameserver. **TXT:** text/verification/security data.

Example: if `api.example.com` is not cached, the resolver can ask the root service about .com, the .com TLD service about example.com, and the authoritative nameserver for the final record.

7. DNS Caching and TTL

Cache: a stored copy used to avoid repeated retrieval or computation. **DNS TTL:** the time, in seconds, that a DNS record may normally remain cached before refresh according to DNS rules.

Example: an A record with TTL 3600 can normally be reused by a recursive resolver for about one hour before a fresh lookup is needed.

Do not confuse: DNS TTL, HTTP Cache-Control: max-age, and CDN object TTL are separate mechanisms even though all deal with freshness/retention.

Benefits of caching

- Lower latency
- Less network traffic
- Lower server/resolver load
- Better scalability
- Trade-off: stale data can be served until refresh/invalidation rules take effect

8. TCP, Ports and Port Binding

TCP is a transport-layer protocol providing reliable, ordered byte-stream communication, retransmission and congestion control.

Port identifies a transport-layer service endpoint. Common conventions are port 80 for HTTP and 443 for HTTPS.

Port binding: a server process can bind/listen on an address and port such as `0.0.0.0:443`. A client normally uses an ephemeral source port to connect to the server destination port.

A TCP connection is commonly described by (source IP, source port, destination IP, destination port) plus the transport protocol.

Port	Common service	Meaning
80	HTTP	HTTP by convention
443	HTTPS	HTTPS/TLS by convention
21	FTP control	Traditional FTP control channel
25	SMTP	Traditional mail transfer

Port numbers are conventions, not permanent ownership: software can be configured to listen on other ports.

9. TLS Handshake and HTTPS

TLS provides confidentiality, integrity and authentication. **HTTPS** is HTTP carried over TLS.

A certificate binds a hostname/identity to a public key and is validated through a trust chain. The certificate does not itself encrypt all application traffic; modern TLS normally uses public-key cryptography for authentication/key establishment and symmetric cryptography for bulk data.

Simplified TLS 1.3 flow

- 1 **ClientHello:** supported versions, cipher suites, random value, key share and extensions.
- 2 **ServerHello:** selected parameters and key share.
- 3 **Certificate:** server provides its certificate chain.
- 4 **CertificateVerify:** server proves possession of the private key.
- 5 **Key derivation:** typically ephemeral Diffie-Hellman establishes shared secrets and forward secrecy.
- 6 **Finished:** both sides verify the handshake.
- 7 **Encrypted application data:** HTTP traffic is protected with symmetric cryptography.

HTTPS example: `https://example.com:443/`. The scan shows an `http://...:443` example; technically the normal matching scheme for port 443 is `https://`.

10. CDN — Content Delivery Network

A **CDN** is a distributed set of edge locations that can cache and serve content closer to users. The scan illustrates an origin such as AWS US-EAST-1 and an edge closer to India, with example latency values.

Component	Role
Origin	Original/authoritative source
Edge	Nearby CDN location serving cached content
Cache hit	Fresh permitted copy is already at edge
Cache miss	Edge fetches from origin and may store the response

A shorter physical/network path can reduce latency and CDN caching can reduce origin load. CDNs may also provide TLS termination, WAF and DDoS-related services depending on configuration.

Important: personalized or highly dynamic responses require careful cache rules so one user's data is not served to another.

11. URL Breakdown

General URL structure: `scheme://[userinfo@]host[:port]/path[?query][#fragment]`

Part	Example	Meaning
Scheme	<code>https</code>	Access scheme/protocol
Subdomain	<code>api</code>	Host label often used for a service
Registered domain	<code>example.com</code>	Registered domain under a TLD
TLD	<code>.com</code>	Top-level domain
Host	<code>api.example.com</code>	Hostname resolved through DNS
Port	<code>443</code>	Transport destination port
Path	<code>/v1/student</code>	Server resource/path
Query	<code>rollnumber=1</code>	Parameters in the request target
Fragment	<code>CGPA</code>	Client-side reference; not sent in HTTP request

Fragment: the browser handles the part after `#`; it is not included in the HTTP request target sent to the server.

API → dynamic work: the scan's phrase is shorthand. An API is an interface/contract for software communication; a web API commonly processes HTTP requests dynamically.

12. API and HTTP Request Structure

API (Application Programming Interface) defines how software components communicate. A web API exposes endpoints that accept HTTP requests and return HTTP responses.

Request parts

- Method
- Request target/path/query
- Headers
- Optional body

Example request

```
GET /api/v1/students/42 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer <token>
```

Possible response

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":42, "name":"Ali", "cgpa":3.7}
```

Path parameter example: /students/42. Query example: /students?page=2&limit;=20. Body example: JSON sent with POST/PUT/PATCH.

13. HTTP Methods / Verbs

HTTP calls these **methods**; “verbs” is a common teaching term.

Method	Meaning	Example
GET	Retrieve a representation	GET /students/42
HEAD	GET-like request without response body	HEAD /students/42
POST	Submit data for processing; commonly create	POST /students
PUT	Create/replace target representation	PUT /students/42
PATCH	Partial modification	PATCH /students/42
DELETE	Remove target resource/association	DELETE /students/42

GET

“Give me this information.” Example: GET /students/42 returns student data. GET is safe and idempotent by HTTP semantics.

HEAD

“Give me the headers.” Useful for metadata such as validators or content length without transferring the response body.

POST

“Submit/create/process this.” POST /students may create a new student. Repeating it can create multiple resources.

PUT

“Replace/create this target.” PUT /students/42 with the same complete representation is intended to produce the same target state each time.

PATCH

“Update this part.” PATCH /students/42 with {“cgpa”:3.8}. PATCH is not inherently idempotent.

DELETE

“Delete this.” Repeating DELETE is idempotent in intended state effect even if the second response is different, such as 404.

Other methods

OPTIONS	Ask capabilities; important for CORS preflight
TRACE	Diagnostic loopback method; often disabled
CONNECT	Establish a tunnel, commonly through a proxy

14. Safe, Idempotent and Cacheable

Method	Safe	Idempotent	Cacheable — general rule
GET	Yes	Yes	Yes
HEAD	Yes	Yes	Yes
PUT	No	Yes	No
POST	No	No	Usually not by default
PATCH	No	Not inherently	No
DELETE	No	Yes	No

Safe: the method's intended semantics are read-only. Server logs/metrics can still be side effects.

Idempotent: repeating the same request has the same intended effect on server state as doing it once. It does *not* mean the response must be identical every time.

Cacheable: a response is permitted to be stored/reused when HTTP rules allow it. It does not mean a cache must store it.

POST nuance: the simple lecture table marks POST non-cacheable. In actual HTTP, some POST responses can be cached when explicit freshness/validation rules permit; shared caches usually do not cache POST by default.

Retry warning: idempotent semantics do not guarantee that every real-world side effect is harmless. Application-level idempotency keys are often used to make retried POST operations safer.

15. HTTP Headers

Headers carry metadata about the request or response.

Header	Purpose
Host	Requested hostname
Accept	Formats client can receive
Content-Type	Format of request/response body
Authorization	Credentials/token
Cookie	Cookies sent by browser
Set-Cookie	Server asks browser to store a cookie
Cache-Control	Caching rules such as max-age
ETag	Representation validator
If-None-Match	Conditional request using ETag
Location	Another URI, commonly after create/redirect
Content-Length	Body size where applicable
User-Agent	Client software information
Origin	Origin information for security/CORS decisions

Conditional request example

If-None-Match: "abc123"

If the stored representation is still current, the server can respond with 304 Not Modified, allowing the client to reuse its cached body.

16. HTTP Response and Status Codes

Status code classes: **1xx information**, **2xx success**, **3xx redirection/cache control**, **4xx client/request issue**, **5xx server/gateway issue**. A 3xx is not necessarily an error.

Class	Meaning	Examples
1xx	Informational	100 Continue; 101 Switching Protocols; 103 Early Hints
2xx	Success	200; 201; 202; 204; 206
3xx	Redirection/control	301; 302; 303; 304; 307; 308
4xx	Client/request issue	400; 401; 403; 404; 405; 409; 422; 429
5xx	Server/gateway issue	500; 501; 502; 503; 504

2xx

200	OK	Successful request
201	Created	New resource created
202	Accepted	Accepted for asynchronous processing
204	No Content	Successful response with no body
206	Partial Content	Requested range returned

3xx

301	Permanent redirect
302	Temporary/common redirect
303	See Other
304	Not Modified — cached representation can be reused
307	Temporary redirect preserving method
308	Permanent redirect preserving method

4xx

400	Bad Request	Malformed/invalid request
401	Unauthorized	Missing/invalid authentication credentials
403	Forbidden	Access refused
404	Not Found	Resource not found
405	Method Not Allowed	Method unsupported for resource
409	Conflict	Conflict with current state
415	Unsupported Media Type	Unsupported body format
422	Unprocessable Content	Semantic/validation problem
429	Too Many Requests	Rate limit exceeded

5xx

500	Internal Server Error	Generic/unhandled server failure
501	Not Implemented	Functionality unsupported
502	Bad Gateway	Invalid upstream response
503	Service Unavailable	Temporarily unable to serve
504	Gateway Timeout	Upstream response timed out

401 vs 403: 401 is about authentication credentials; 403 is refusal to authorize/access. “401 = wrong login, 403 = logged in but forbidden” is a useful shortcut but not the complete rule.

17. HTTP 101 — Switching Protocols

The lecture calls protocol switching “rare.” **101 Switching Protocols** means the server is switching to a protocol requested by the client.

Classic WebSocket over HTTP/1.1 uses an HTTP request with `Upgrade: websocket` and `Connection: Upgrade`. If accepted, the server responds with `101 Switching Protocols` and the connection changes to WebSocket framing.

This is uncommon for ordinary CRUD APIs but important for persistent bidirectional communication.

18. Web Caching

Caching is storing a copy so a system can reuse it rather than retrieving/recomputing it every time.

Common cache layers

- Browser cache
- DNS/resolver cache
- CDN edge cache
- Reverse proxy cache
- Application cache
- Database-related caches

Cache hit vs miss

Hit: usable copy is found. **Miss:** no usable copy exists, so data must be fetched or computed.

HTTP example

```
Cache-Control: max-age=3600
ETag: "abc123"
```

Caching improves performance but creates freshness/invalidation challenges. `Vary` can affect the cache key, and `Expires` is an older freshness mechanism using an absolute date.

19. REST — Representational State Transfer

REST means *Representational State Transfer*. It is an architectural style associated with Roy Fielding, not a protocol.

Core constraints

- Client-server separation
- Stateless client-server interaction
- Cacheable responses
- Uniform interface
- Layered system
- Code-on-demand is optional

Resource examples

```
GET /students/42
PATCH /students/42
DELETE /students/42
GET /students/42/courses
```

A resource can have representations such as JSON or XML. A “student 42” representation might be `{"id":42,"name":"Ali","cgpa":3.7}`.

REST vs HTTP: HTTP is a protocol. REST is an architectural style. Many RESTful APIs use HTTP, but the terms are not synonyms.

Statelessness: every request carries the information necessary to process it; the server should not depend on conversational state from earlier requests. Cookies/tokens can carry identity, but a server-side session means server-side session state exists.

20. SOAP vs REST vs GraphQL

Technology	What it is	Typical characteristics
SOAP	Protocol/messaging framework	XML messages, formal contracts such as WSDL, enterprise integrations
REST	Architectural style	Resources, representations, standard HTTP methods/status codes commonly used
GraphQL	API query language + runtime	Typed schema; client requests selected fields; often one endpoint for queries/mutations

SOAP: structured XML messaging with formal contracts commonly described using WSDL; can use different transports.

REST: commonly resource-oriented and uses HTTP semantics such as methods, status codes, headers and caching.

GraphQL: clients specify the fields they want from a typed schema. It is neither REST nor simply “REST with different syntax.”

21. Authentication vs Authorization

Authentication = Who are you? It verifies identity. **Authorization = What are you allowed to do?** It checks permissions.

Stage	Example
1. Credentials	Username/email + password
2. Authentication	Server verifies credentials
3. Identity state	Session or token is created
4. Request	Cookie or Authorization header is sent
5. Authorization	Role/resource permission is checked
6. Response	Allowed request succeeds; otherwise 401/403 as appropriate

Example: a user can successfully authenticate but still be forbidden from editing another user's record. Authentication alone does not grant every permission.

Password security: passwords should not be stored as plaintext. Password-hashing/KDF options include Argon2id, bcrypt, scrypt and PBKDF2.

22. Cookies, Sessions, JWT and Client State

Cookies: browser-managed values sent in matching requests. Servers set them with `Set -Cookie`; browsers return them in `Cookie`. Important attributes include `Secure`, `HttpOnly`, `SameSite`, `Domain`, `Path` and `Max-Age`.

Session: commonly server-side state identified by a session ID stored in a cookie. The server uses the ID to retrieve the user's session.

JWT: JSON Web Token, a compact token containing claims and commonly a signature. A signed JWT is not automatically encrypted; its payload should not be treated as secret.

Typical header: `Authorization: Bearer <token>`

Client-side state: state can be held in memory, cookies or browser storage. Security depends on the storage mechanism and threats such as XSS and CSRF.

Lecture clarification: cookies, sessions and JWTs are mechanisms for carrying identity/context across requests. JWT use does not automatically make an API RESTful; a server-side session introduces server-side state.

23. WebSocket

WebSocket provides a persistent, bidirectional communication channel. Once established, both client and server can send messages without creating a new HTTP request for every message.

HTTP/REST style	WebSocket
Request → response	Persistent connection
Client normally initiates requests	Either side can send after connection
Good for CRUD/resources	Good for live events/streams
HTTP caching is important	Caching is not the main model

Common uses: chat, live notifications, collaborative editing, live dashboards and games.

24. React / Angular and the Frontend Note

The scan mentions React/Angular near state and communication. They are frontend technologies, not networking protocols.

Technology	Role
React	JavaScript library/ecosystem for building UI components
Angular	Frontend framework for web applications
REST API	Server/application interface commonly called by frontend code
WebSocket	Real-time bidirectional communication

A React/Angular application may call a REST API using fetch or another HTTP client and may open a WebSocket for real-time updates. The scan's "Particle Animation, font cloud effect" appears to be an ancillary UI/frontend note rather than a networking concept.

25. End-to-End Worked Example

```
https://api.example.com:443/v1/students/42?include=gpa#profile
```

1. URL

Scheme=https; host=api.example.com; subdomain=api; registered domain=example.com; TLD=.com; port=443; path=/v1/students/42; query=include=gpa; fragment=profile.

2. DNS

Browser/OS checks cached information. If needed, a recursive resolver obtains the answer through DNS hierarchy. A/AAAA records provide IPv4/IPv6 addresses.

3. Transport

HTTP/1.1 or HTTP/2 normally uses TCP; HTTP/3 uses QUIC over UDP.

4. TLS

HTTPS causes TLS negotiation. The certificate is checked for the hostname and shared keys are established.

5. HTTP request

```
GET /v1/students/42?include=gpa HTTP/1.1
Host: api.example.com
Accept: application/json
```

Notice that #profile is not sent to the server.

6. Server

The server may authenticate the caller, authorize access, consult cache/database, and generate a response.

7. Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":42, "name":"Ali", "cgpa":3.7}
```

8. Client

The application processes the response. The browser may use the fragment for client-side navigation and may cache the response if permitted.

26. Exam Revision Sheet

One-line definitions

Term	Remember
Internet	Global network of interconnected networks
Web	Application system running on the Internet
DNS	Distributed naming system
Resolver	Performs DNS lookup for a client
IPv4	32-bit addressing
IPv6	128-bit addressing
TCP	Reliable ordered transport
Port	Transport endpoint number
TLS	Secure communication protocol
HTTPS	HTTP over TLS
CDN	Distributed edge caching/delivery
URL	Resource locator with scheme/host/path/query/fragment
API	Interface/contract between software components
HTTP	Application-layer request/response protocol
REST	Architectural style
SOAP	XML messaging protocol/framework
GraphQL	Typed API query language/runtime
Authentication	Identity verification
Authorization	Permission check
JWT	Compact token format
WebSocket	Persistent bidirectional channel

Methods

GET	Retrieve	Safe, idempotent
HEAD	Headers	Safe, idempotent
POST	Submit/create/process	Not safe; not inherently idempotent
PUT	Replace/create target	Idempotent; not safe
PATCH	Partial update	Not inherently idempotent
DELETE	Remove	Idempotent; not safe

Status code memory: **1xx information, 2xx success, 3xx redirect/control, 4xx request/client issue, 5xx server/gateway issue.** Key codes: 200, 201, 204, 301, 302, 304, 400, 401, 403, 404, 500, 502, 503, 504.

27. Technical Clarifications to the Handwritten Notes

These points preserve the lecture's original idea while making the terminology precise.

Lecture shorthand	Precise clarification
Browser → Resolver / Root → TLD → IP → TCP	DNS may be satisfied from cache; otherwise the recursive resolver may query root, TLD and authoritative DNS. Transport starts after IP.
1969 → Internet	1969 is an ARPANET milestone; the modern Internet evolved over many years.
1993 → first browser	Mosaic was a major early graphical browser, not literally the first browser.
http://...:443	Port 443 conventionally matches HTTPS, so use https://... for a normal secure URL.
DNS TTL = all caching	DNS TTL is specifically for DNS records; HTTP max-age and CDN freshness are separate.
POST not cacheable	Usually not cached by shared caches by default, but HTTP allows cacheable POST responses in some explicitly controlled cases.
PATCH not idempotent	PATCH is not inherently idempotent; an individual PATCH design can be idempotent.
401 = authentication; 403 = password wrong	401 concerns missing/invalid authentication; 403 is access refusal. Exact app behavior varies.
REST = protocol	REST is an architectural style, not a protocol.
REST = HTTP	HTTP is a protocol; REST is an architectural style often implemented using HTTP.
JWT = encrypted	JWT payloads are not necessarily encrypted; signed JWTs are common.
Cookies/session/JWT work around statelessness	They carry context/identity between requests. Server-side sessions create server state; JWT does not automatically make an API RESTful.
WebSocket as REST	WebSocket is a separate persistent bidirectional communication mechanism.
React/Angular as networking	They are frontend technologies that can consume APIs and WebSockets.

Final mental model

URL identifies where/how → **DNS** finds an address → **IP** routes packets → **TCP/QUIC** transports data → **TLS** protects HTTPS → **HTTP** carries requests/responses → **cache/CDN** may serve stored content → **API/REST/GraphQL/SOAP** organizes application communication → **authentication/authorization** controls identity/access → **status codes** describe the result.

Source note: This document is based on the uploaded 5-page handwritten lecture scan and expands the concepts visible there. Technical clarifications are included to prevent simplified classroom shorthand from being memorized as an exact protocol rule.